Introduction to NLP — Assignment 1 (Project Phase 1)

Data Acquisition, Text Processing & Cleaning — Amazon Fine Food Reviews

Team name	[TEAM NAME]
Members (name – roll no.)	Abdul Wahab Farukh - L1F23BSCS0311; Muhammad Ibrahim - L1F23BSCS0302; Abeer - [Roll no.]
Repository / tag	[REPO URL] — tag `assignment-1`

1. Topic & dataset

Topic: sentiment / rating prediction on English product reviews. **Dataset:** Amazon Fine Food Reviews (Stanford SNAP, McAuley & Leskovec; Kaggle `snap/amazon-fine-food-reviews`). It holds **568,454** reviews (Oct 1999 – Oct 2012) of 74,258 products by 256,059 users, one 300 MB CSV with 10 columns. Licence: CC0 / public domain as listed on Kaggle. **Why:** it is large, genuinely messy real-world text (HTML, shouting, typos, run-together words) with a built-in label, and it grows naturally into every later phase (BoW/TF-IDF, embeddings, classical ML, deep models). **Target for later phases:** `Score` (1–5) and a derived sentiment label (1–2 negative, 3 neutral, 4–5 positive). The loader script is `src/load_data.py`; the raw data is not committed, and `requirements.txt` reproduces the environment (Python 3.11+, pandas, NLTK, spaCy).

2. Exploration

Loaded with pandas: **568,454** documents; average length **80.3 words** (median 56, max 3,432). The target is strongly skewed: Score 5 = 63.9%, 4 = 14.2%, 3 = 7.5%, 2 = 5.2%, 1 = 9.2% (derived sentiment: 78.1% positive, 14.4% negative, 7.5% neutral), so later phases must handle class imbalance. **174,562 reviews (30.7%) are exact duplicates** (same user, time and text, posted under several product variants); they are removed in cleaning to avoid train/test leakage.

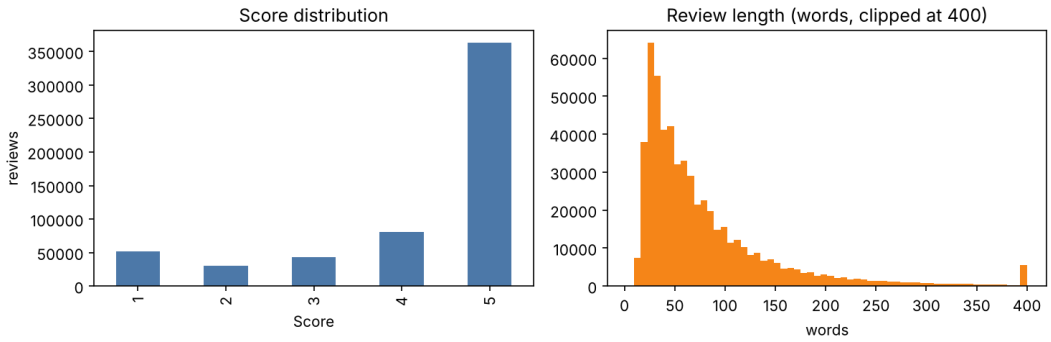


Figure 1 — Score distribution and review length.

On a random sample of 50,000 reviews: **N = 4,030,821 tokens, V = 41,818 types** (type/token ratio 0.0104). **41.1% of types occur exactly once** (17,183 hapax legomena) while the top 100 types cover 54.3% of all tokens — a textbook Zipfian long tail (Figure 2). The observed curve is above the ideal 1/rank line in the head and falls off sharply in the tail, where typos and rare product names live. Note that `br` is the 10th most frequent raw token (57,126 occurrences): an HTML artefact, not language.

```
notebook cell: Task 2 — tokens vs types

Tokens (N) = 4,030,821   Types / vocabulary (V) = 41,818   type/token ratio = 0.0104
Types occurring exactly once (hapax) : 17,183 = 41.1% of the vocabulary
Top 100 types cover 54.3% of all tokens
Top 15: [('the', 163652), ('i', 132911), ('and', 112928), ('a', 108469), ('to', 88975), ('it', 81505), ('of', 70327),
    ('is', 64002), ('this', 58856), ('br', 57126), ('for', 48379), ('in', 48211), ('my', 40887), ('that', 37770),
    ('but', 33094)]
```

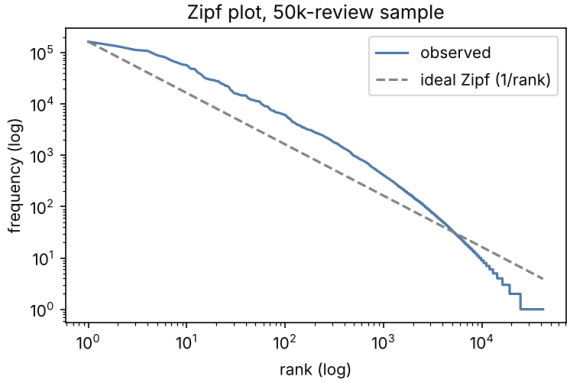


Figure 2 — Zipf (rank–frequency) plot, log–log.

```
notebook cell: Task 2 — dataset statistics

Number of documents          : 568,454
Average length (words)       : 80.3  (median 56, max 3,432)
Missing Summary / ProfileName: 27 / 26
Distinct products / users    : 74,258 / 256,059
Exact duplicate reviews      : 174,562  (same user, time, text)

Target distribution (Score 1-5):
        count  percent
Score
1       52268      9.2
2       29769      5.2
3       42640      7.5
4       80655     14.2
5      363122     63.9

Derived sentiment target:
           count  percent
Score
positive  443777     78.1
negative   82037     14.4
neutral    42640      7.5
```

Messy text examples (real reviews): HTML `<br />` and an anchor tag with URL (Id 22); HTML entity `[…]` (Id 3593); shouting with `! ! !` (Id 155); sentences glued together without a space after the full stop, plus typos like “perfict”, “Youll” (Id 196); non-ASCII “Ît’s” (Id 90).

```
notebook cell: Task 2 — messy examples

--- HTML tags (<br />)  (Id=22)
'I bought these for my husband who is currently overseas. He loves these, and apparently his staff likes them also.<br
    />There are generous amounts of Twizzlers in each 16-ounce bag, and this was well worth the price. <a
   href="http://www.amazon.com/gp/product/B001GVISJM">Twizzlers, Strawberry, 16-Ounce Bags (Pack of 6)</a>'

--- HTML entity / quotes  (Id=3593)
"I don't understand why this dog food is not full of reviews. Checkout [&#8230;] they give this dog food 4 out of 5
   stars which beats science diet and purina one which I use to use and diamond natural is less expensive."

--- URL / anchor tag  (Id=22)
'I bought these for my husband who is currently overseas. He loves these, and apparently his staff likes them also.<br
    />There are generous amounts of Twizzlers in each 16-ounce bag, and this was well worth the price. <a
   href="http://www.amazon.com/gp/product/B001GVISJM">Twizzlers, Strawberry, 16-Ounce Bags (Pack of 6)</a>'

--- Shouting + repeated punctuation  (Id=155)
'AWESOME!!! such a yummy flavor I got it as a healthy alternative to the desserts we normally eat and I am so glad
   that I did there are so many things you can do with Jello desserts and still have them taste good and be good for
   you. I will definitentally be purchasing this product again the flavor was so wonderful.'

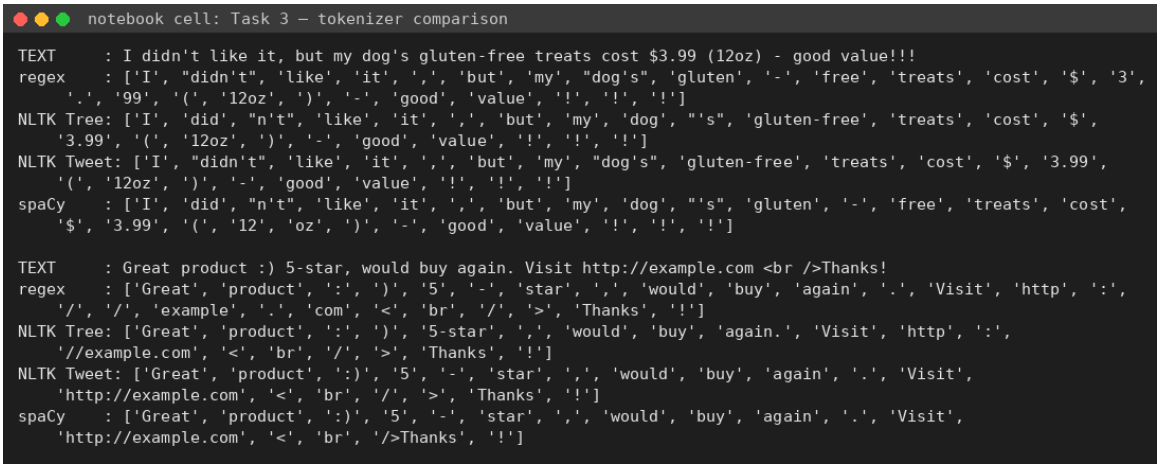
--- Missing space after full stop  (Id=196)
'These little tins of sugar free mine mints were a great deal.There perfict for on the go.Fifty mints per tin is just
   the right amount to cary along.And there safer for your teeth than the ones with sugar.The price of twelve dollars
    was very resonable.Also there great for shareing.If you are looking for a mint try these ones.Youll be glad you
   did.'

--- Non-ASCII characters  (Id=90)
"Great food! I love the idea of one food for all ages & breeds. Ît's a real convenience as well as a really good
   product.  My 3 dogs eat less, have almost no gas, their poop is regular and a perfect consistency, what else can a
    mom ask for!!"
```

3. Tokenization & regular expressions

Methods compared on the same sentences and on 5,000 reviews: (1) our regex tokenizer `\w+(?:'\w+)?|[^\w\s]`, (2) NLTK *Treebank*, (3) NLTK *Tweet*, (4) spaCy tokenizer. They differ on exactly the phenomena this data contains:

Phenomenon	regex	Treebank	Tweet	spaCy
don't	don't	do + n't	don't	do + n't
gluten-free	gluten - free	gluten-free	gluten-free	gluten - free
$3.99	$ 3 . 99	$ 3.99	$ 3.99	$ 3.99
12oz	12oz	12oz	12oz	12 oz
:)	:)	:)	:)	:)
"taste.Would"	taste . Would	taste.Would (not split)	—	—
tokens/review (5k)	99.4	94.5	97.1	96.4
types (5k)	19,236	24,695	20,552	21,795



Regular expressions written for this data (match counts over all 568,454 reviews):

Pattern	Regex	Explanation	Reviews
HTML tags	`<[^>]+>`	'<', then any run of non-'>' characters, then '>' → , <a href=…>	147,324 (25.9%)
URLs	`(?:https?://|www\.)\S+`	http(s):// or www. followed by non-space characters	11,788 (2.1%)
Prices	`\$\s?\d+(?:[.,]\d+)*`	literal $, optional space, digits, optional .99 / ,000 groups	24,308 (4.3%)
Repeated punct.	`([!?.])\1{1,}`	a mark followed by itself (back-reference) → !!!, …, ??	103,439 (18.2%)
Quantity + unit	`\b\d+(?:\.\d+)?\s?(?:oz|lb|lbs|ct|count|pack|g|kg|ml|l)\b`	number with a pack-size unit → 12oz, 2 lb, 24 count	31,802 (5.6%)
HTML entities	`&(?:#\d+|[a-z]+);`	& then #digits or letters then ; → " "	1,856 (0.3%)

Tokenization challenges found in this data. (1) Contractions and negation: stripping punctuation naively turns “didn't” into “didnt” (or “didn” + “t”), hiding the negation that decides sentiment; Treebank/spaCy split it into “did” + “n't”, the regex keeps it whole. We therefore expand contractions ourselves (`n't` → `not`) before tokenizing. **(2) Hyphenation:** “gluten-free”, “low-carb”, “5-star”, “k-cups” are one concept; the regex and spaCy split them while Treebank/Tweet keep them. We split them (both halves are meaningful words and the vocabulary shrinks). **(3) Missing spaces / HTML residue:** “taste.Would buy again.Highly recommend” stays glued under Treebank (“taste.Would”) but splits correctly with the regex, and un-removed HTML shows up as the fake words “br”, “href”.

4. Cleaning pipeline

One function, `clean_text()` in `src/cleaning.py`, applies the steps below in this fixed order; running it twice gives identical output. Order matters: Unicode is normalized first so later regexes see canonical characters; HTML/URLs are removed *before* case-folding and tokenizing so they do not become junk tokens; contractions are expanded before tokenizing so negations survive; stop words are removed after that so negations are protected; lemmatization is last because it needs clean word forms.

#	Step	Justification for this data
1	Unicode NFC + curly quotes/dashes → ASCII	Canonical form before any regex; some reviews contain non-ASCII characters (e.g. “Ît's”).
2	Noise removal: html.unescape, tags, URLs, e-mails, prices, repeated !?.	26% of reviews contain HTML; entities like " and … appear; prices/URLs carry no sentiment.
3	Case folding (lower-case)	Merges “Great/great/GREAT” (vocabulary 586,944 raw → 111,150 lower-case letter tokens). Capitalisation is a mild emotion signal, so we **keep it as features** `caps_ratio` and `n_exclaim` computed from the raw text.
4	Contraction expansion (n't→not, 've→have …)	Keeps negation as an explicit token “not”.
5	Tokenize: letters-only regex `[a-z]+`	Drops digits, punctuation, sizes; leaves clean word tokens.
6	Stop words: spaCy list **minus** negations / intensifiers (no, not, never, nothing, very, too, but, only, just, really …)	Task is sentiment: the default list deletes words that flip or scale polarity (see screenshot).
7	Lemmatization (spaCy lookup lemmatizer + small override table)	Chosen over stemming (see below). Overrides fix better/best/worse and cookies.
8	Whitespace normalization on join	Single spaces; reviews empty after cleaning are dropped (1 review).

```
notebook cell: Task 4 — step-by-step cleaning

1 NFC/quotes   : I DIDN'T like it!!! <br />Paid $3.99 at http://x.com &quot;Gluten-Free&quot; cookies were better than
     the BEST ones I've tried...
2 noise removal: I DIDN'T like it!  Paid   at   "Gluten-Free" cookies were better than the BEST ones I've tried.
3 case folding : i didn't like it!  paid   at   "gluten-free" cookies were better than the best ones i've tried.
4 contractions : i did not like it!  paid   at   "gluten-free" cookies were better than the best ones i have tried.
5 tokens       : ['i', 'did', 'not', 'like', 'it', 'paid', 'at', 'gluten', 'free', 'cookies', 'were', 'better',
    'than', 'the', 'best', 'ones', 'i', 'have', 'tried']
6 stop words   : ['not', 'like', 'paid', 'gluten', 'free', 'cookies', 'better', 'best', 'ones', 'tried']
7 lemmas       : ['not', 'like', 'pay', 'gluten', 'free', 'cookie', 'better', 'best', 'one', 'try']
clean_text()   : not like pay gluten free cookie better best one try
```

```
notebook cell: Task 4 — stop-word check

spaCy default list size : 326 | our list size: 291
negation/intensity words in spaCy default (would be deleted): ['no', 'not', 'nor', 'never', 'none', 'nothing',
    'nobody', 'neither', 'cannot', 'without', 'very', 'too', 'but', 'only', 'just', 'really']
still removed by our list                                  : []

Before/after with default vs our list:
  default : good buy nice
  ours    : not good never buy again but very nice
```

Stemming vs lemmatization — we apply lemmatization. The Porter stemmer produces non-words (“delici”, “amazingli”, “happili”, and “wa” from “was”) that would make Phase 2 vocabularies, embeddings and error analysis hard to read, and it also merges words with different polarity-relevant meaning. Lemmatization keeps real words (“bought→buy”, “cookies→cookie”, “tastes→taste”) and leaves “delicious”, “better” and “best” intact. Limitations: the lookup lemmatizer has no part-of-speech context (e.g. “canned→can”), so some errors remain.

```
notebook cell: Task 4 — stemming vs lemmatization

         word Porter stem      lemma
      cookies       cooki     cookie
      tasting        tast      taste
       tastes        tast      taste
    delicious      delici  delicious
  disappointed  disappoint disappoint
disappointing  disappoint disappoint
       bought      bought        buy
       better      better     better
         best        best       best
        worse        wors      worse
      flavors      flavor     flavor
    flavorful      flavor  flavorful
   amazingly   amazingli  amazingly
    packaged      packag    package
   packaging      packag    package
     unsalted      unsalt   unsalted
      happily     happili    happily
         dogs         dog        dog
       eating        eat        eat
          was          wa         be
```

Vocabulary size before vs after cleaning (on the de-duplicated corpus of 393,892 reviews): raw whitespace tokens **586,944** → lower-cased letter tokens 111,150 → **90,596** after the full pipeline, a reduction of **84.6%**; tokens fall from 31.4M to 15.2M. The remaining 90k types are dominated by typos, brand names and rare words (the long tail).

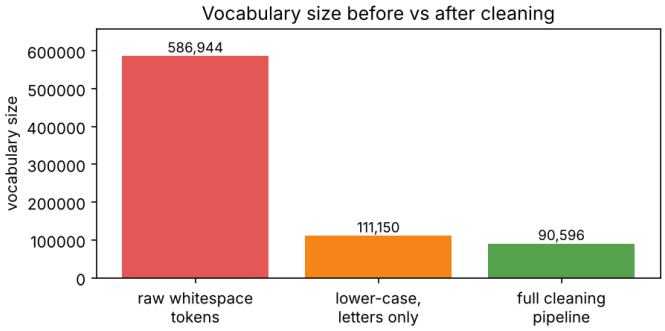
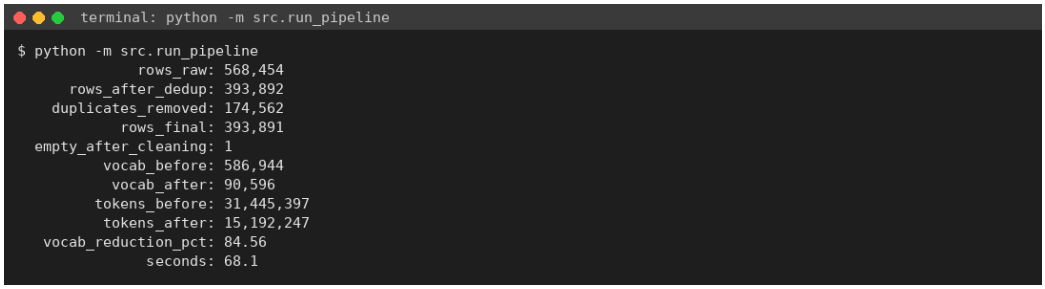


Figure 3 — Vocabulary size at each stage.



5. Reproducibility

Run steps (also in the README): `git clone <repo> && git checkout assignment-1` → create a Python 3.11+ virtual environment → `pip install -r requirements.txt` → place `Reviews.csv` in `data/raw/` → `python -m src.run_pipeline`. This writes `data/processed/cleaned_reviews.parquet` (393,891 rows: ids, Score, sentiment, raw and cleaned Summary/Text, caps_ratio, n_exclaim) which Phase 2 loads directly. The exploration notebook `notebooks/01_exploration_and_cleaning.ipynb` re-runs end to end and reproduces every number in this report. A fresh clone, new virtual environment, `pip install -r requirements.txt` and run was tested: it reproduced exactly the same 393,891-row cleaned file (identical contents) and the same vocabulary statistics. Exact tagged commit: **[REPO URL]/tree/assignment-1**.

6. Contribution statement

Member	Contribution
Abdul Wahab Farukh	Dataset selection, data-loading script, README, repo setup
Muhammad Ibrahim	Exploration notebook, Zipf analysis, messy-text examples, tokenization comparison and regular expressions
Abeer	Cleaning pipeline, stop-word and lemmatization experiments, report